מבוא למדעי המחשב - 67101 - סמסטר ב' 21/22
מרצה: ד"ר אריה שלזינגר
מועד א' - 26.6.22

<u>חלק א' - הצעות לפתרון:</u>

א'1.

```
def waze(x, y):

    if x > y:
        return 0
    elif x == y:
        return 1
    else:
        return waze(add1(x), y) + waze(twice(x), y)
```

ב'1.

```
waze(min(x, y), z) >= waze(max(x, y), z)

waze(x, z) >= waze(x, y) * waze(y, z)
```

א'2.

```
    def delete_at_loc(self, del_loc):
        cur = self.head
        for i in range(1, del_loc):
            cur = cur.next
        cur.next = cur.next.next
        self.len -= 1
```

ב'2.

```
    def find_node_loc(self, data):# returns node and location
        cur = self.next
        loc = 0
        while cur != None:
            if cur.data == data:
                return cur, loc
            else:
                cur = cur.next
                loc = loc + 1
        return None
```

3.א' סמנו את כל הביטויים הנכונים (g מוגדרת להיות פונקציה כלשהי של n).

1. $O(2^n) = O(2^n * n^2)$ **False**
2. $O(5n) = O(n/5)$ **True**
3. $O(\lg(2^n)) = O(\lg(2^n * n^2))$ **True**
4. $O(g(n)+2/n) = O(g(2n))$ **False**

3.ב' בחרו את סיבוכיות זמן הריצה של הפונקציה f4 כפונקציה של אורך הרשימה lst.

```
def f4(lst):
    x, n = 0, len(lst)
    for t in range(n):
        k = 1
        while k < t:
            x += lst[k]
            k *= 3
```

Solution:

O(N*logN)

4.א'

```
def go_flat(lst):
    res = []
    templist = [lst]
    while templist:
        cur = templist.pop()
        if isinstance(cur, list):
            templist.extend(cur)
        else:
            res.insert(0, cur)
    print(res)
```

4.ב'

```
def go_flat_gen(lst):
    for k in lst:
        if isinstance(k,list):
            yield from go_flat_gen(k)
        else:
            yield k
```

חלק ב' - הצעות לפתרון:

.5

```
def monotonic(t):
    if t.is_a_leaf():
        return True
    for subtr in t.branches: # for each subtree
        if t.data > subtr.data or not f2(subtr):
            return False
    return True
```

.6

```
def get_brackets(st): # st is a string
    brack = "(){}[]"
    if st == "":
        return ""
    elif st[0] in brack:
        return st[0] + get_brackets(st[1:])
    else:
        return get_brackets(st[1:])

def brackets_match(ch1, ch2):
    return ch1 == "{" and ch2 == "}" or (
           ch1 == "(" and ch2 == ")" or
           ch1 == "[" and ch2 == "]" )

def isBalanced(st):
    result = isBalanced_Helper(st)
    return result == ""

def isBalanced_Helper(brackets):
    if brackets == "":
        return brackets
    else:
        ch1 = brackets[0]
        subSt = isBalanced_Helper(brackets[1:])

        if len(subSt) > 0:
            ch2 = subSt[0]
            if brackets_match(ch1, ch2):
                return subSt[1:]
```

```
        return ch1 + subSt

def balanced(str1)::
    bracks = get_brackets(str1)
    return isBalanced(bracks)
```

.7

```
def plus_it(t):
    if t.data == '+':
        return sum([plus_it(x) for x in t.branches])
    elif t.data == '*':
        res = 1
        for k in t.branches:
            exp , res = res , 0
            for i in range(plus_it(k)):
                res = res + exp
        return res
    else:
        return t.data
```

.8

```
def de_comp(n, base_lst):
    if n == 0:
        return Lnk.empty
    elif base == Lnk.empty:
        raise ArithmeticError
    elif base.data > n:
        return de_comp(n, base_lst.next)
    else:
        return Lnk(base.data, de_comp(n - base_lst.data,base_lst.next))
```

.9 הצעה לפתרון רקורסיבי:

```
def max_sum(liz1, liz2, liz3, n):
    def max_sum_rec(liz1, liz2, liz3, sum):
        blockIt = set() # temporay block access to a list
        if sum > n:
            return 0
        elif liz1 == blockIt:
            return sum
        results = []
```

```
        for num in liz1:
            results.append(max_sum_rec(liz2, liz3, blockIt, sum + num))
        return max(results)

    return max_sum_rec(liz1, liz2, liz3, 0)
```

הצעה לפתרון לא רקורסיבי:

```
l1 = [3, 1, 2] ; l2 = [5, 2, 8, 11, 1] ; l3 = [6, 0]
lim = 30
def max_sums(l1,l2,l3,lim):
    lt = [(i,j,k,sum((i,j,k))) for i in l1 for j in l2 for k in l3 if
sum((i,j,k))<=lim]
    try:
        fit = min(lt,key=lambda x:lim-x[-1])
        for i in range(len(fit)-1):
            print(fit[i], end=" ")
    except ValueError:
        print("No can do")

max_sums(l1,l2,l3,lim)
# for limit = 30 =>  3, 11, 6
# for limit =  1 => "No can do"
```

.10

```
from math import gcd
def lcm(a, b): return abs(a // gcd(a, b) * b) #intentionally fails\
                                              #when gcd(a,b)==0

class Frac:

    def __init__(self, mone, mahane):
        self.mone = mone

        if mahane == 0:
            raise ZeroDivisionError

        self.mahane = mahane
        self._reduce()
    ######### DO NOT EDIT ABOVE THIS LINE! #########
```

```
def _reduce(self):
    frac_gcd = gcd(self.mone, self.mahane)
    self.mone = self.mone // frac_gcd
    self.mahane = self.mahane // frac_gcd

    if self.mone < 0 and self.mahane < 0:
        self.mone = abs(self.mone)
        self.mahane = abs(self.mahane)
    elif self.mone > 0 and self.mahane < 0:
        self.mone = (-1) * self.mone
        self.mahane = abs(self.mahane)

def __add__(self, other):
    mone = self.mone*other.mahane + self.mahane*other.mone
    mahane = self.mahane * other.mahane
    return Frac(mone, mahane)

def __sub__(self, other):
    mone = self.mone*other.mahane - self.mahane*other.mone
    mahane = self.mahane * other.mahane
    return Frac(mone, mahane)

def __mul__(self, other):
    mone, mahane = self.mone * other.mone, self.mahane * \
                   other.mahane
    return Frac(mone, mahane)

def __truediv__(self, other):
    if other.mone == 0:
        raise ZeroDivisionError
    mone, mahane = self.mone*other.mahane, self.mahane*other.mone
    return Frac(mone, mahane)


def __eq__(self, other):
    if isinstance(other, Frac):
        return (self.mone == other.mone) and \
               (self.mahane == other.mahane)
    else:
        return False



def __ge__(self, other):
```

```
        if isinstance(other, Frac):
            return self.mone/ self.mahane >= other.mone / other.mahane
        else:
            return False

    def __str__(self):
        if abs(self.mone) > self.mahane:
            int_part = abs(self.mone) // self.mahane
            mone_part = abs(self.mone) - (self.mahane * int_part)
            if self.mone < 0:
                int_part = (-1)*int_part
            return str(int_part) + " " + str(mone_part) + "/" + \
                   str(self.mahane)
        if self.mone < self.mahane:
            return str(self.mone) + "/" + str(self.mahane)
        if self.mone == self.mahane:
            return "1"

    def __pow__(self, power):
        if power>=0:
            mone, mahane = pwr(self.mone, self.mahane, power)
        else:
            mone, mahane = pwr(self.mahane, self.mone,(-1)*power)

        return Frac(mone, mahane)


def pwr(a, b, power):
    if power == 0:
        return 1, 1
    tmp1, tmp2 = 1, 1

    for i in range(power):
        tmp1 *= a
        tmp2 *= b

    return tmp1,tmp2
```